SOFTWARE REQUIREMENTS SPECIFICATION

CampusConnect: Student Academic & Communication Portal

Prepared in IEEE 830-style SRS structure

Submitted in partial fulfillment of the requirements for the
Bachelor of Technology in Computer Science & Engineering

Submitted to:
Rajasthan Technical University, Kota (Rajasthan)

Project Guide
Er. Ram Babu Buri

Department of Computer Science & Engineering
Arya College of Engineering & I.T.
SP-42, RIICO Industrial Area, Kukas, Jaipur

Version 1.0 | Academic Session 2026–27

Table of Contents

- Document Control
- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features and Functional Requirements
- 5. Non-Functional Requirements
- 6. Logical Database Requirements
- 7. Verification and Acceptance Criteria
- Appendix A. Use Case Summary
- Appendix B. Current Scope Versus Planned Scope
- Appendix C. Open Issues Before Final Baseline

Document Control

Field	Value
Project	CampusConnect: Student Academic & Communication Portal
Document	Software Requirements Specification (SRS)
Version	1.0
Prepared for	Advanced Java Lab / Project-Based Learning
Institution	Arya College of Engineering & I.T., Jaipur
University	Rajasthan Technical University, Kota
Project Guide	Er. Ram Babu Buri
Session	2026–27
Status	Requirements baseline for the current college-focused release

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for CampusConnect, a web-based Student Academic & Communication Portal being developed initially for Arya College of Engineering & I.T., Jaipur. It describes the system scope, users, interfaces, functional behaviour, quality requirements, constraints, and acceptance criteria. It is intended for the project team, project guide, evaluators, testers, and future maintainers.

This document follows the traditional IEEE 830-style SRS organization. Requirements distinguish the current development focus from planned capabilities; a feature must not be considered implemented merely because it is specified here.

1.2 Scope

CampusConnect aims to provide students and teachers with a common web-based entry point for institution-related information. The current release focuses on registration, login, STUDENT and TEACHER roles, profile-related functionality, dashboard navigation, authentication security, and MySQL persistence. The wider product vision includes announcements, assignments, chat, club management, academic information, and eventual multi-college support.

The first deployment target is one college. Multi-college support is a future architectural direction and must not be treated as production-ready until institution-scoped data ownership and cross-institution access controls are implemented and tested.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface used by the frontend to communicate with backend operations.
BCrypt	Password-hashing algorithm used to store password verifiers rather than plaintext passwords.
CORS	Cross-Origin Resource Sharing; browser rules controlling which origins may call a server.
CRUD	Create, Read, Update, and Delete operations.
JWT	JSON Web Token; a signed token format used for authentication claims.
RBAC	Role-Based Access Control; permissions based on an authenticated account's role.
REST	Architectural style for exposing operations through HTTP APIs.
SRS	Software Requirements Specification.
TEACHER	Faculty account intended for teacher-facing workflows.
STUDENT	Student account intended for student-facing workflows.
UI	User Interface.
XSS	Cross-Site Scripting, a vulnerability involving attacker-controlled scripts executing in a browser.

1.4 References

- CampusConnect Project Report, “CampusConnect: Student Academic & Communication Portal,” Academic Session 2026–27.
- React Documentation: https://react.dev/

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- MySQL Documentation: https://dev.mysql.com/doc/
- Apache Maven Documentation: https://maven.apache.org/guides/
- JSON Web Token (JWT), RFC 7519: https://www.rfc-editor.org/rfc/rfc7519
- Axios Documentation: https://axios-http.com/
- Tailwind CSS Documentation: https://tailwindcss.com/docs
- Git Documentation: https://git-scm.com/doc

1.5 Overview

Section 2 describes the product context, major functions, user classes, operating environment, constraints, assumptions, and dependencies. Section 3 lists external interface requirements. Section 4 specifies functional requirements. Section 5 specifies non-functional requirements. Section 6 records logical data requirements and conceptual entities. Section 7 defines verification and acceptance criteria. Appendices summarize use cases and the distinction between current scope and planned extensions.

2. Overall Description

2.1 Product Perspective

CampusConnect is a browser-based application with a React frontend, an Axios API client, a Java Spring Boot backend, Spring Security components, and a MySQL relational database. The frontend sends HTTP/JSON requests to REST controllers. Controllers delegate business operations to services; persistence components access MySQL. Security filters and backend authorization are responsible for protecting restricted operations.

2.2 Product Functions

- Register an account after validating required information.
- Authenticate registered users through a login endpoint.
- Distinguish STUDENT and TEACHER roles and apply role-aware behaviour.
- Create and validate authentication tokens according to the configured JWT implementation.
- Present a dashboard after successful authentication.
- Support user profile viewing and updates where the relevant operations are implemented.
- Persist account and related data in MySQL.
- Attach a bearer token to API requests when available and clear stale client-side session state after a 401 response.
- Provide a foundation for future announcements, assignments, chat, club management, academic sections, and institution-scoped operation.

2.3 User Classes and Characteristics

User class	Description	Access principle
STUDENT	Student using student-facing portal features and permitted profile functions.	May access only permitted student data and operations.
TEACHER	Faculty member using teacher-facing dashboard and future academic workflows.	May access authorised teacher functions and academic data.
COLLEGE ADMIN (future)	Institution-level administration if implemented.	Restricted to explicitly authorised accounts and the assigned institution.
PLATFORM ADMIN (future)	Potential platform-level institution onboarding and configuration.	Cross-institution access only where explicitly authorised.

2.4 Operating Environment

- Client: modern web browser on a desktop or mobile device.
- Frontend: React.js with Vite tooling and Tailwind CSS; Axios for API requests.
- Backend: Java with Spring Boot and Maven.
- Security: Spring Security, JWT-related services, and BCrypt password encoding, subject to verification in the current branch.
- Database: MySQL, configured for the campusconnect database in the development environment.
- Development tools: Git/GitHub and a code editor.

2.5 Design and Implementation Constraints

- The project is developed within an academic PBL timeline.
- The initial target is Arya College of Engineering & I.T., Jaipur.
- Database credentials and JWT secrets must be supplied through environment-specific configuration and must not be committed to source control.
- Backend authorization must not depend only on frontend visibility or client-supplied role labels.
- The exact physical database schema and API payload fields must be verified against the current source repository.
- Planned modules must be labelled as planned until their implementation and tests are confirmed.
- Multi-college operation requires explicit institution ownership and service/repository-level enforcement.

2.6 Assumptions and Dependencies

- MySQL Server is installed, running, and reachable with a correctly configured application account.
- Java, Maven, Node.js, and npm versions are compatible with the project configuration.
- The frontend API base URL and backend port are configured consistently.
- Required environment variables or local configuration values are supplied before startup.
- The repository README and current source code are authoritative for exact commands, DTO fields, entity mappings, and enabled security behaviour.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide registration and login screens, dashboard navigation, and profile-related views appropriate to the authenticated role. Forms should label fields clearly, indicate validation errors in text, preserve non-sensitive inputs after validation failures where practical, and display useful success or failure feedback. The interface should be responsive on common desktop and mobile viewport sizes.

3.2 Hardware Interfaces

No dedicated hardware interface is required. Users access the system through a device capable of running a supported web browser and connecting to the application server.

3.3 Software Interfaces

Interface	Requirement
React frontend to backend	HTTP requests carrying JSON payloads to configured REST API endpoints.
Authentication API	POST /api/auth/register creates an account; POST /api/auth/login authenticates an existing account. Exact request and response fields must match the current DTOs.
API client	Axios shall use the configured backend base URL and attach a Bearer token when available, according to the current implementation.
Backend to database	Spring datasource/JDBC configuration shall connect to MySQL using the configured URL, username, password, and driver.
Security components	Spring Security and JWT-related components shall validate authentication and enforce access rules where enabled in the current build.

3.4 Communications Interfaces

The frontend and backend communicate over HTTP during local development and should use HTTPS in a deployed environment. Requests and responses use JSON where applicable. CORS shall allow only the origins required by the deployment. Credentials, password hashes, signing secrets, and full tokens shall not be exposed in ordinary error responses or logs.

4. System Features and Functional Requirements

The following requirements are uniquely identified for traceability. “Shall” indicates a required behaviour for the specified release. Requirements that depend on a future module are explicitly marked future or conditional.

4.1 Registration

FR-001 — The system shall provide a registration operation for creating a new user account.

FR-002 — The system shall validate required registration fields on the backend, regardless of frontend validation.

FR-003 — The system shall reject registration when an email address already exists within the configured uniqueness scope.

FR-004 — The system shall validate the role-specific identifier, such as enrollment number for a student or employee ID for a teacher, where required by the current registration model.

FR-005 — The system shall encode a submitted password using the configured password encoder before persistence.

FR-006 — The system shall not return a plaintext password or password hash in an API response.

FR-007 — The system shall commit account creation only when the registration transaction completes successfully.

4.2 Login and Authentication

FR-008 — The system shall provide a login operation for registered users.

FR-009 — The backend shall verify submitted credentials against the stored password hash.

FR-010 — On successful authentication, the backend shall return the configured authentication response, including a token when token issuance is enabled.

FR-011 — The system shall reject invalid credentials without creating an authenticated session.

FR-012 — The system shall validate the signature and expiration of JWTs on protected requests when JWT protection is enabled.

FR-013 — The frontend shall store and attach the authentication token according to the implemented client-side session strategy.

FR-014 — When the frontend receives a 401 Unauthorized response, it shall clear stale client-side token state and redirect the user to the login screen, consistent with the current client implementation.

4.3 Role-Based Access

FR-015 — The system shall distinguish STUDENT and TEACHER accounts.

FR-016 — The backend shall enforce role permissions for protected operations; hiding a UI element alone shall not be treated as authorization.

FR-017 — Public registration shall not allow a client to grant itself an unauthorised elevated role.

FR-018 — The system shall deny protected API requests that lack valid authentication where authentication is required.

FR-019 — The system shall return an appropriate authorization failure when an authenticated user lacks permission for an operation.

4.4 User Profile

FR-020 — The system shall associate profile information with the authenticated user account where profile functionality is implemented.

FR-021 — The system shall validate profile fields before saving changes.

FR-022 — Profile read/update operations shall derive the account identity from the authenticated principal rather than trusting an arbitrary user identifier supplied by the client.

FR-023 — Student and teacher profile fields may differ according to role; exact fields shall match the implemented data model.

4.5 Dashboard

FR-024 — After successful authentication, the frontend shall navigate the user to the dashboard flow.

FR-025 — The dashboard shall present information and actions appropriate to the authenticated role.

FR-026 — The dashboard shall not display private data that the current user is not authorised to view.

4.6 Database and Persistence

FR-027 — The system shall persist account information in the configured MySQL database.

FR-028 — The system shall maintain data consistency using appropriate database constraints and transactional service behaviour where applicable.

FR-029 — The system shall keep database credentials outside public source control.

4.7 Future Academic and Communication Features

FR-030 — A future announcements module should allow authorised users to publish notices for a defined audience.

FR-031 — A future assignments module should support teacher-created assignments and due dates, with student access controlled by the relevant academic scope.

FR-032 — A future chat module should restrict conversations and message access to authorised participants.

FR-033 — A future club-management module should support club information and membership workflows.

FR-034 — Before multi-college use, each user and institution-owned record shall be associated with a trusted institution identifier, and cross-institution access shall be denied unless explicitly authorised.

5. Non-Functional Requirements

5.1 Security

NFR-SEC-001 — Passwords shall be stored as one-way password hashes and shall not be stored or logged in plaintext.

NFR-SEC-002 — JWT signing secrets and database credentials shall be provided through environment-specific configuration.

NFR-SEC-003 — Protected endpoints shall validate authentication and enforce server-side authorization.

NFR-SEC-004 — Error responses shall not expose stack traces, SQL statements, password hashes, signing secrets, or sensitive filesystem details.

NFR-SEC-005 — The deployed application should use HTTPS to protect data in transit.

NFR-SEC-006 — CORS configuration shall be restricted to required frontend origins.

NFR-SEC-007 — Institution-scoped authorization and negative cross-college access tests shall be completed before multi-college operation.

5.2 Usability and Accessibility

NFR-USE-001 — Forms shall provide clear labels and understandable validation feedback.

NFR-USE-002 — Primary screens shall remain usable at common desktop and mobile viewport sizes.

NFR-USE-003 — Interactive elements should support keyboard operation with visible focus indicators.

NFR-USE-004 — Errors shall be communicated in text and shall not rely on colour alone.

5.3 Reliability and Availability

NFR-REL-001 — The application shall provide controlled error handling for invalid input, authentication failures, and database connectivity problems.

NFR-REL-002 — The project shall document repeatable startup steps for the frontend, backend, and database.

NFR-REL-003 — Persisted account data shall remain available after backend restart, provided the database remains available and persistence has been correctly configured.

5.4 Performance

NFR-PER-001 — Database queries and indexes should be designed to support expected academic data volumes.

NFR-PER-002 — No numeric performance target is asserted in this specification because the project report provides no measured baseline. A response-time target should be agreed and tested before release.

5.5 Maintainability and Scalability

NFR-MNT-001 — The backend shall separate controller, service, persistence, and configuration responsibilities.

NFR-MNT-002 — New modules should be added without placing business or database logic in frontend components.

NFR-SCL-001 — The design should support future institution-scoped data, but multi-college operation shall not be enabled until isolation controls are implemented and verified.

5.6 Portability and Configuration

NFR-PORT-001 — Setup documentation shall identify required Java, Maven, MySQL, Node.js, and npm tooling.

NFR-CONF-001 — Environment-specific configuration shall be documented without publishing real credentials or signing secrets.

NFR-CONF-002 — The configured frontend API address, backend port, datasource URL, and allowed CORS origins shall be consistent.

6. Logical Database Requirements

The following is a conceptual data model derived from the project report. It is not a declaration that every listed table or field already exists in the current database. Physical names, types, constraints, and relationships must be checked against the repository's entity classes and schema.

Entity	Conceptual attributes	Purpose / constraints
User	id, name, email, password_hash, role	Account identity and authentication. Email uniqueness and allowed roles must be enforced.
Student Profile	user_id, enrollment_no, department, semester	Student-specific information; identifier uniqueness scope must be defined.
Teacher Profile	user_id, employee_id, department, designation	Teacher-specific information; employee identifier must be validated.
Institution (future)	id, name, code, status	Required for institution ownership before multi-college operation.

Entity	Conceptual attributes	Purpose / constraints
Department (conceptual)	id, institution_id, department_name	Organizes academic data within an institution.
Announcement (planned)	id, institution_id, title, body, created_by, audience, created_at	College communication; requires author and audience authorization.
Assignment (planned)	id, institution_id, subject_id, title, due_date, created_by	Academic task distribution; access should be scoped to the relevant class/course.

6.1 Data Integrity Requirements

- Primary identifiers shall uniquely identify records.
- Required fields shall be validated before persistence.
- Passwords shall be stored only as encoded hashes.
- Role-specific profile records shall be consistent with the associated user role.
- Institution-owned records shall reference a trusted institution identifier before multi-college use.
- Deletion, archival, and retention rules should be defined before production deployment.

7. Verification and Acceptance Criteria

The tests below are planned verification activities. Actual outcomes must be recorded after execution; no test is marked as passed merely because it appears in this SRS.

Test ID	Scenario	Expected result
TC-01	Register with valid required fields	Account is created once and a safe response is returned.
TC-02	Register with an existing email	Request is rejected without creating a duplicate account.
TC-03	Register with missing required fields	Validation error is returned.
TC-04	Login with correct credentials	Authentication succeeds and response matches the frontend contract.
TC-05	Login with incorrect password	Authentication fails and no token/session is issued.
TC-06	Call a protected API without a token	Request is rejected where authentication is required.
TC-07	Call a protected API with an invalid or expired token	Request is rejected.
TC-08	Student attempts a teacher-only operation	Backend denies the action when such an operation exists.
TC-09	Restart backend and log in again	Persisted account can be authenticated when the database is available.
TC-10	Stop MySQL and start the backend	Startup/connectivity failure is diagnosable without leaking secrets.
TC-11	Receive a 401 response in frontend	Stale token is cleared and the user is redirected to login.
TC-12	User from one institution requests another institution's data (future)	Access is denied before multi-college operation.

7.1 Release Acceptance

- The backend builds and starts using documented steps.
- The frontend starts and communicates with the backend.
- Registration and login success and failure paths are tested.
- Password hashing and token behaviour are checked against the current code.
- MySQL persistence is verified across a backend restart.
- Role restrictions are tested at the backend.
- Profile and dashboard behaviour is verified for each supported role.
- Known limitations and unimplemented future modules are clearly documented.

Appendix A. Use Case Summary

Use case	Primary actor	Precondition	Outcome
Register account	Student / Teacher	Registration page is available.	Account is created if input is valid and unique.
Log in	Student / Teacher	Account exists.	Valid credentials establish the configured authenticated flow; invalid credentials are rejected.
View dashboard	Student / Teacher	Authentication succeeds.	Role-appropriate dashboard is displayed.
View/update profile	Student / Teacher	User is authenticated and profile operation exists.	Permitted profile data is displayed or updated.
Access protected API	Authenticated user	Valid authentication token is present where required.	Request is allowed only if authentication and authorization checks pass.
Publish announcement (planned)	Authorised teacher/admin	Announcements module is implemented.	Notice is stored and visible to its permitted audience.
Create assignment (planned)	Authorised teacher	Assignments module is implemented.	Assignment is stored with validated academic scope and due date.

Appendix B. Current Scope Versus Planned Scope

Area	Status in project report	SRS treatment
Registration and login APIs	Reported as implemented; must be reverified in the current branch.	Specified as core functional requirements.
JWT-related security and BCrypt	Reported as present; active enforcement and runtime behaviour require verification.	Specified with verification caveat.
Axios token handling and dashboard flow	Reported as implemented.	Specified as client/session requirements.
Profile read/update	Scope identified; exact completion should be checked in repository.	Conditional requirements pending verification.
Announcements, assignments, chat, clubs	Planned modules.	Future requirements only; not treated as completed.
Multi-college support	Long-term direction; institution isolation not yet claimed complete.	Must be implemented and tested before multi-college use.

Appendix C. Open Issues Before Final Baseline

- Confirm the exact request and response DTO field names for registration and login.
- Confirm the current database entity classes, schema, and profile relationships.
- Verify that JWT signing, validation, and Spring Security rules are actively enforced in the current branch.
- Verify profile read/update endpoints and their role permissions.
- Record actual test results and attach genuine screenshots from the running application.
- Set measurable performance targets if required by the final deployment context.
- Confirm institution-level data ownership and isolation before adding a second college.